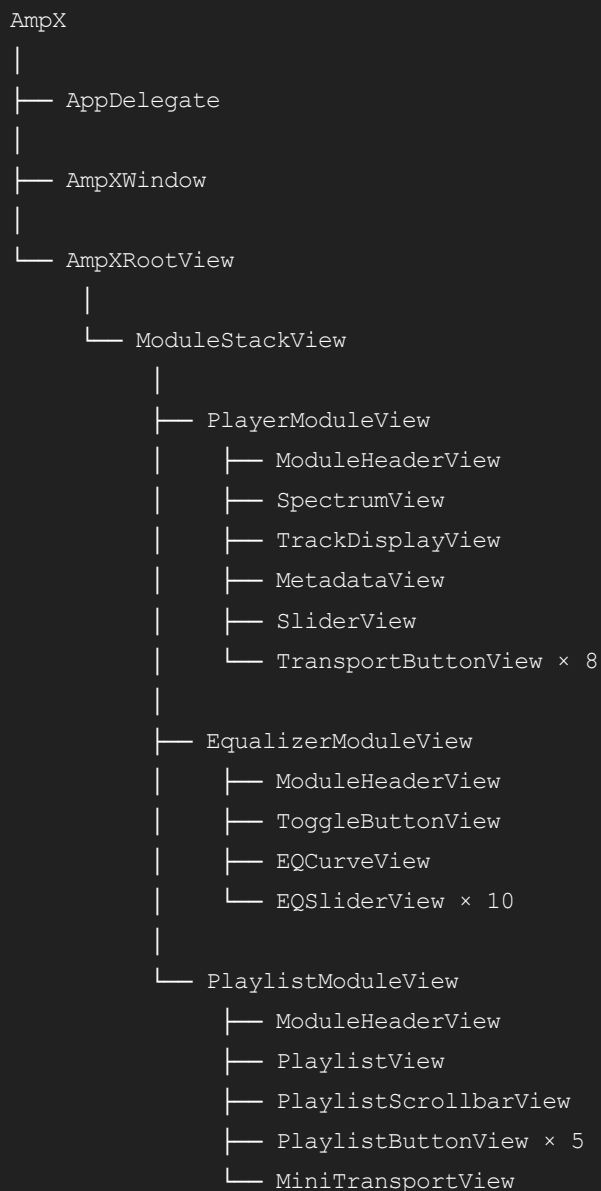


Yes. For **AmpX on macOS**, if your goal is to reproduce that latest mockup *very closely*, I would **not build the UI primarily in SwiftUI**.

I would use **AppKit + Core Animation + custom `NSView` drawing**, with SwiftUI only around the edges if you want it later.

The reason is simple: the design depends on exact borders, 1 px highlights, bevels, compact spacing, custom sliders, custom scrollbars, and fixed proportions. AppKit gives you much more deterministic control.

Recommended architecture



The important part is:

Each module is a self-contained custom AppKit view.

That maps naturally to your stackable architecture.

1. Don't use standard macOS controls

This is probably the most important implementation decision.

Don't do:

Swift

```
NSButton
NSSlider
NSTextField
NSScrollView
```

with their standard appearance.

Instead:

Swift

```
AmpXButton
AmpXSlider
AmpXLabel
AmpXScrollView
AmpXWindow
```

where you control the rendering.

For example:

Swift

```
final class AmpXButton: NSView {
    var title: String = ""
    var isActive = false
    var isPressed = false

    override func draw(_ dirtyRect: NSRect) {
        // Draw entire button yourself
    }

    override func mouseDown(with event: NSEvent) {
        isPressed = true
        needsDisplay = true
    }

    override func mouseUp(with event: NSEvent) {
        isPressed = false
        needsDisplay = true
    }
}
```

This gives you much more control over the visual result.

2. Use a fixed design coordinate system

The generated design is approximately:

998 × 1576

at 2×.

So I'd define your design dimensions as:

Swift

```
enum AmpXMetrics {  
    static let designWidth: CGFloat = 499  
    static let moduleGap: CGFloat = 5  
  
    static let playerHeight: CGFloat = 165  
    static let equalizerHeight: CGFloat = 200  
    static let playlistHeight: CGFloat = 290  
}
```

The important thing is that **499 × ~788 is the canonical logical size**.

Don't start with a responsive layout and hope it eventually looks like the image.

Start with the image's geometry.

3. Window

I'd make the initial window approximately:

Swift

```
let contentSize = NSSize(  
    width: 499,  
    height: 788  
)
```

And:

Swift

```
window.styleMask = [
    .titled,
    .closable,
    .miniaturizable,
    .resizable
]
```

However, I'd actually hide the normal macOS title bar.

You want AmpX itself to own the chrome.

Swift

```
window.titleVisibility = .hidden
window.titlebarAppearsTransparent = true
```

Then:

AmpX custom title/header
Player
Equalizer
Playlist

4. Use Core Graphics for the actual skin

I'd create a reusable renderer.

Swift

```
struct AmpXStyle {

    static let background =
        NSColor(calibratedRed: 0.043,
                green: 0.059,
                blue: 0.094,
                alpha: 1)

    static let panel =
        NSColor(calibratedRed: 0.082,
                green: 0.106,
                blue: 0.161,
                alpha: 1)
```

```
static let panelLight =
    NSColor(calibratedRed: 0.125,
            green: 0.157,
            blue: 0.227,
            alpha: 1)

static let border =
    NSColor(calibratedRed: 0.231,
            green: 0.275,
            blue: 0.361,
            alpha: 1)

static let borderHighlight =
    NSColor(calibratedRed: 0.396,
            green: 0.443,
            blue: 0.529,
            alpha: 1)

static let green =
    NSColor(calibratedRed: 0.0,
            green: 1.0,
            blue: 0.196,
            alpha: 1)

static let yellow =
    NSColor(calibratedRed: 1.0,
            green: 0.824,
            blue: 0.10,
            alpha: 1)

static let orange =
    NSColor(calibratedRed: 1.0,
            green: 0.616,
            blue: 0.0,
            alpha: 1)

static let display =
    NSColor.black
}
```

I would keep **all colors centralized**.

That makes skinning later trivial.

5. Roboto Mono

Bundle:

```
RobotoMono-Regular.ttf
RobotoMono-Medium.ttf
RobotoMono-SemiBold.ttf
```

inside your app.

Then register it at startup.

Swift

```
import CoreText

func registerFonts() {
    let fonts = [
        "RobotoMono-Regular",
        "RobotoMono-Medium",
        "RobotoMono-SemiBold"
    ]

    for name in fonts {
        guard let url = Bundle.main.url(
            forResource: name,
            withExtension: "ttf"
        ) else { continue }

        CTFontManagerRegisterFontsForURL(
            url as CFURL,
            .process,
            nil
        )
    }
}
```

Then:

Swift

```
let font = NSFont(
    name: "Roboto Mono",
    size: 13
)!
```

I'd use Roboto Mono for **everything**:

```
AmpX
EQUALIZER
PLAYLIST
128 kbps
48 kHz
```

```
track names
buttons
playlist
EQ labels
timestamps
```

That's one of the defining characteristics of the new design.

6. Build a reusable module frame

This is probably the most valuable custom component.

Swift

```
final class ModuleView: NSView {

    let contentView = NSView()

    var title: String = ""
    var showBrand = true

    override func draw(_ dirtyRect: NSRect) {
        super.draw(dirtyRect)

        guard let ctx = NSGraphicsContext.current?.cgContext else {
            return
        }

        let rect = bounds.insetBy(dx: 1, dy: 1)

        // Outer shadow
        ctx.setShadow(
            offset: CGSize(width: 0, height: -2),
            blur: 5,
            color: NSColor.black.withAlphaComponent(0.45).CGColor
        )

        ctx.setFillColor(
            AmpXStyle.panel.cgroundColor
        )

        ctx.fill(rect)

        ctx.setShadow(offset: .zero, blur: 0)

        // Outer border
        ctx.setStrokeColor(
            AmpXStyle.border.cgroundColor
        )

        ctx.stroke(rect)
```

```

        // Top/left highlight
        let highlight = rect.insetBy(dx: 2, dy: 2)

        ctx.setStrokeColor(
            AmpXStyle.borderHighlight
                .withAlphaComponent(0.5)
                .CGColor
        )

        ctx.stroke(highlight)
    }
}

```

Then every module gets the same frame.

7. Module header

The header should be its own view.

Swift

```

final class ModuleHeaderView: NSView {

    var brand = "AmpX"
    var moduleName = ""

    override func draw(_ dirtyRect: NSRect) {

        guard let ctx =
            NSGraphicsContext.current?.cgContext else {
            return
        }

        let y = bounds.midY

        // Yellow decorative lines
        drawAccentLine(
            ctx,
            from: 18,
            to: 150,
            y: y
        )

        drawAccentLine(
            ctx,
            from: bounds.width - 150,
            to: bounds.width - 48,
            y: y
        )
    }
}

```

```

        drawText(
            brand,
            at: CGPoint(x: bounds.midX - 40, y: y - 7),
            fontSize: 16,
            color: AmpXStyle.text
        )

        if !moduleName.isEmpty {
            drawText(
                moduleName,
                at: CGPoint(x: bounds.midX + 18, y: y - 6),
                fontSize: 13,
                color: AmpXStyle.text
            )
        }
    }
}

```

You'd tune those positions from the reference.

8. Don't use Auto Layout everywhere

For this particular interface, I'd use **explicit geometry**.

Something like:

Swift

```

override func layout() {

    player.frame = CGRect(
        x: 0,
        y: height - playerHeight,
        width: width,
        height: playerHeight
    )

    equalizer.frame = CGRect(
        x: 0,
        y: height - playerHeight - gap - equalizerHeight,
        width: width,
        height: equalizerHeight
    )

    playlist.frame = CGRect(
        x: 0,
        y: 0,
        width: width,

```

```
        height: playlistHeight
    )
}
```

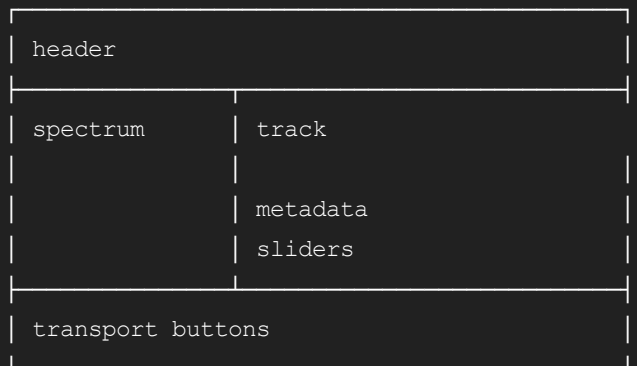
Why?

Because the interface is essentially a **precision instrument panel**.

Auto Layout isn't buying you much here.

9. Player geometry

I'd divide the player approximately:



Something like:

Swift

```
let spectrumWidth = width * 0.37
let rightWidth = width * 0.63
```

10. Spectrum display

Make this a custom Core Graphics view.

Swift

```
final class SpectrumView: NSView {

    var levels: [CGFloat] =
        Array(repeating: 0, count: 24)

    override func draw(_ dirtyRect: NSRect) {

        guard let ctx =
```

```

        UIGraphicsContext.current?.cgContext else {
            return
        }

        ctx.setFillColor(NSColor.black.cgColor)
        ctx.fill(bounds)

        drawGrid(ctx)

        for (index, level) in levels.enumerated() {

            let barWidth: CGFloat = 5
            let spacing: CGFloat = 2

            let x = 8 +
                CGFloat(index) *
                (barWidth + spacing)

            let height =
                max(2, level * (bounds.height - 12))

            let rect = CGRect(
                x: x,
                y: 5,
                width: barWidth,
                height: height
            )

            drawSpectrumBar(ctx, rect)
        }
    }
}

```

Use discrete rectangular bars.

Do not use a smooth waveform.

11. Spectrum colors

You can calculate the color from the bar height.

Swift

```

if level > 0.75 {
    color = AmpXStyle.yellow
} else if level > 0.45 {
    color = AmpXStyle.green
} else {
    color = AmpXStyle.green
}

```

```
.withAlphaComponent(0.65)  
}
```

The overall impression should be:

```
yellow  
yellow-green  
green  
dark green
```

12. Track display

This should be another custom view.

Swift

```
final class TrackDisplayView: NSView {  
  
    var text = "4. Crusher-P - Echo (3:50)"  
  
    override func draw(_ dirtyRect: NSRect) {  
  
        let rect = bounds.insetBy(dx: 1, dy: 1)  
  
        NSColor.black.setFill()  
        rect.fill()  
  
        AmpXStyle.border.setStroke()  
        rect.stroke()  
  
        let attributes: [NSAttributedString.Key: Any] = [  
            .font: NSFont(  
                name: "Roboto Mono Medium",  
                size: 13  
            )!,  
            .foregroundColor: AmpXStyle.green  
        ]  
  
        text.draw(  
            at: CGPoint(x: 10, y: bounds.midY - 7),  
            withAttributes: attributes  
        )  
    }  
}
```

13. Custom AmpX buttons

This is where I'd spend a lot of time.

The latest generated image's buttons can be improved even further by making the bevel **subtle and geometric**.

Swift

```
final class AmpXButton: NSView {

    var label = ""
    var icon: AmpXIcon?
    var active = false

    private var pressed = false

    override func draw(_ dirtyRect: NSRect) {

        guard let ctx =
            NSGraphicsContext.current?.cgContext else {
            return
        }

        let r = bounds.insetBy(dx: 1, dy: 1)

        // Main surface
        ctx.setFillColor(
            AmpXStyle.panelLight.cgColor
        )

        ctx.fill(r)

        // Top / left highlight
        ctx.setStrokeColor(
            AmpXStyle.borderHighlight.cgColor
        )

        ctx.move(to: CGPoint(x: r.minX, y: r.maxY))
        ctx.addLine(to: CGPoint(x: r.maxX, y: r.maxY))
        ctx.strokePath()

        // Bottom / right dark edge
        ctx.setStrokeColor(
            AmpXStyle.borderDark.cgColor
        )

        ctx.stroke(r)

        // Pressed state
        if pressed {
            drawPressedState(ctx, r)
        }
    }
}
```

```
        drawContents(ctx)
    }
}
```

The important thing:

No standard `NSButton` appearance.

14. Button states

Implement:

Swift

```
enum ButtonState {
    case normal
    case hover
    case pressed
    case active
    case disabled
}
```

Then render each state.

Normal

dark steel

Hover

slightly lighter steel

Pressed

inset

Active

green indicator

This will make the buttons feel significantly better than the old Winamp controls.

15. Button dimensions

Main controls:

~44 × 40

Secondary controls:

~64 × 32

Small controls:

~28 × 28

Don't make everything the same size.

The latest mockup works because there's a clear hierarchy:

PLAYBACK



SECONDARY



UTILITY



16. Icons

I would **not** use SF Symbols.

They'll immediately make it feel like a macOS application rather than AmpX.

Create a tiny custom icon renderer:

Swift

```
enum AmpXIcon {  
    case play  
    case pause  
    case stop  
    case previous  
    case next  
    case eject  
    case shuffle
```

```

    case repeat
    case menu
    case minimize
    case maximize
    case close
}

```

Then:

Swift

```

func drawIcon(
    _ icon: AmpXIcon,
    in rect: CGRect,
    context ctx: CGContext
)

```

Use basic Core Graphics paths.

For example:

Swift

```

case .play:
    ctx.move(to: ...)
    ctx.addLine(to: ...)
    ctx.addLine(to: ...)
    ctx.closePath()
    ctx.fillPath()

```

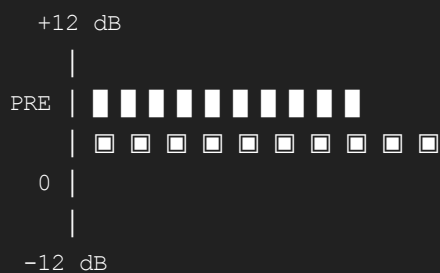
This gives you complete visual consistency.

17. Equalizer

The EQ should be almost entirely custom-drawn.

ON AUTO PRESETS

—— EQ curve ——



I'd make:

Swift

```
final class EqualizerView: NSView
```

and store:

Swift

```
var bands: [Float] =  
    Array(repeating: 0, count: 10)
```

18. EQ sliders

Don't use `NSSlider`.

Create:

Swift

```
final class EQSliderView: NSView {  
  
    var value: CGFloat = 0  
  
    override func draw(_ dirtyRect: NSRect) {  
        drawTrack()  
        drawFill()  
        drawThumb()  
        drawLabel()  
    }  
  
    override func mouseDown(with event: NSEvent) {  
        updateValue(event)  
    }  
  
    override func mouseDragged(with event: NSEvent) {  
        updateValue(event)  
    }  
}
```

Then map:

```
-12 → 0  
0 → center
```

19. EQ slider rendering

The track should be approximately:



The thumb is a small metallic rectangle.

Not a circular modern knob.

20. Playlist

I'd make the playlist a custom view rather than `NSTableView`.

That gives you exact control.

Swift

```
final class PlaylistView: NSView {

    var tracks: [Track] = []
    var selectedIndex: Int?

    override func draw(_ dirtyRect: NSRect) {

        drawBackground()

        for (index, track) in tracks.enumerated() {
            drawRow(
                track,
                index: index
            )
        }

        drawScrollbar()
    }
}
```

21. Playlist row

Calculate manually:

Swift

```
let rowHeight: CGFloat = 22
```

Then:

Swift

```
let y =  
    bounds.height -  
    CGFloat(index + 1) * rowHeight
```

Selected:

Swift

```
NSColor(  
    calibratedRed: 0.13,  
    green: 0.18,  
    blue: 0.29,  
    alpha: 1  
)
```

Normal:

black

Text:

green

Selected:

light gray

22. Align track durations

This is where Roboto Mono is particularly useful.

You don't need complicated column layout.

You can simply draw:

```
"4. Crusher-P - Echo"
```

and right-align:

```
"3:50"
```

using:

```
Swift
```

```
let durationWidth: CGFloat = 42
```

That will produce the characteristic terminal-like appearance.

23. Custom scrollbar

Again, don't use the native macOS scrollbar.

Draw:



The arrows can be amber.

The thumb steel blue-gray.

24. Playlist footer

Build another reusable component:

```
Swift
```

containing:

ADD REM SEL MISC

|< ▶ || ■ >|

0:00/27:45

LIST

OPTS

Again, all custom rendered.

25. Stackable modules

This part should be independent from rendering.

Create:

Swift

```
protocol AmpXModule {
    var moduleID: String { get }
    var preferredHeight: CGFloat { get }
    var isCollapsed: Bool { get set }
}
```

Then:

Swift

```
final class ModuleStackView: NSView {

    var modules: [NSView] = []

    func addModule(_ module: NSView) {
        modules.append(module)
        addSubview(module)
        needsLayout = true
    }

    func moveModule(
        from source: Int,
        to destination: Int
    ) {
        let module = modules.remove(at: source)
```

This means you can eventually do:

Player
EQ
Playlist

or:

Playlist
Player
EQ

or:

```
Player
Playlist
```

without the module implementation knowing anything about stacking.

26. Module dragging

Make the header the drag handle.

On mouse down:

Swift

```
beginDrag()
```

During drag:



The insertion marker should be **AmpX yellow**.

27. High-DPI rendering

This is extremely important given your original complaint.

Do **not** design around physical pixels.

Use points:

Swift

```
499 × 788 points
```

Core Animation/AppKit will render them appropriately at:

1×

2×

3×

For thin lines, however, you need to account for the backing scale.

You can query:

Swift

```
window?.backingScaleFactor
```

and snap geometry accordingly.

A helper is useful:

Swift

```
func pixelAlign(
    _ value: CGFloat,
    scale: CGFloat
) -> CGFloat {
    round(value * scale) / scale
}
```

That prevents fuzzy 1 px borders.

28. Rendering hierarchy

I'd use this division:

Core Animation

For:

- shadows
- opacity
- animations
- module movement
- hover transitions

Core Graphics

For:

- borders
- buttons
- EQ
- spectrum
- playlist
- icons
- decorative lines

AppKit

For:

- window
- mouse/keyboard events
- menus
- accessibility
- clipboard
- file dialogs

That is a very good fit for AmpX.

29. Animation

Keep it extremely restrained.

For example:

Swift

```
NSAnimationContext.runAnimationGroup { context in
    context.duration = 0.12
```

Dragging modules:

150–200ms

Button press:

60–80ms

EQ:

50–100ms

The application should feel responsive rather than animated.

30. Window resizing

There is one tricky decision.

If the user resizes the window from:

499 × 788

to:

700 × 1100

I would **scale the UI proportionally only to a point**, rather than making all text huge.

For example:

Swift

```
let scale = min(  
    width / 499,  
    height / 788  
)
```

Then constrain:

```
minimum scale = 0.85  
maximum scale = 1.35
```

This preserves the compact AmpX character.

31. Even better: separate logical and visual dimensions

Define:

Swift

```
struct AmpXLayout {

    static let width: CGFloat = 499

    static let player = CGRect(
        x: 0,
        y: 0,
        width: 499,
        height: 165
    )

    static let equalizer = CGRect(
        x: 0,
        y: 170,
        width: 499,
        height: 200
    )

    static let playlist = CGRect(
        x: 0,
        y: 375,
        width: 499,
        height: 290
    )
}
```

Then let the module stack calculate the actual positions.

This gives you a clean separation between:

DESIGN

and:

RUNTIME WINDOW

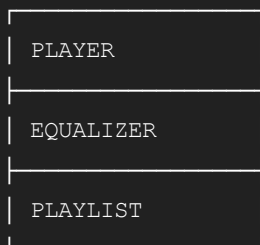
32. The most important thing: don't make the first implementation too abstract

I'd actually recommend **not** starting with a giant design system.

Build this sequence:

Phase 1

Get this exact static composition:



with no audio functionality.

Phase 2

Make every visual component custom.

Phase 3

Add mouse interactions.

Phase 4

Add real audio state.

Phase 5

Add stacking/docking.

Phase 6

Add skins/themes.

This will make visual iteration much faster.

33. Suggested project structure

I'd start the Xcode project like this:

```
AmpX/  
|  
├─ App/  
|   ├─ AppDelegate.swift  
|   └─ AmpXWindowController.swift
```

- |
- |— Theme/
 - |— AmpXStyle.swift
 - |— AmpXMetrics.swift
 - |— AmpXFonts.swift
- |
- |— Components/
 - |— AmpXButton.swift
 - |— AmpXSlider.swift
 - |— AmpXLabel.swift
 - |— ModuleHeaderView.swift
 - |— ModuleView.swift
- |
- |— Modules/
 - |— PlayerModule.swift
 - |— EqualizerModule.swift
 - |— PlaylistModule.swift
- |
- |— Player/
 - |— SpectrumView.swift
 - |— TrackDisplayView.swift
 - |— TransportControls.swift
- |
- |— Equalizer/
 - |— EQCurveView.swift
 - |— EQBandSlider.swift
 - |— EQPresetView.swift
- |
- |— Playlist/
 - |— PlaylistView.swift
 - |— PlaylistRow.swift
 - |— PlaylistScrollbar.swift
 - |— PlaylistToolbar.swift
- |
- |— Modules/
 - |— ModuleStackView.swift
 - |— ModuleDragController.swift
 - |— ModuleLayout.swift
- |
- |— Audio/
 - |— AudioEngine.swift
 - |— PlayerState.swift
 - |— PlaylistState.swift
 - |— EqualizerState.swift
- |
- |— Resources/
 - |— Fonts/
 - |— RobotoMono-Regular.ttf
 - |— RobotoMono-Medium.ttf
 - |— RobotoMono-SemiBold.ttf

34. What I would tell your coding agent

You can essentially give it this:

Implement the AmpX macOS UI using AppKit and Core Graphics.

Do not use SwiftUI controls, NSSlider, NSButton, NSTableView, native macOS scrollbars, or SF Symbols for the primary interface.

The visual reference is a high-DPI evolution of classic Winamp.

Canonical design size:

499 × 788 logical points.

The interface consists of three vertically stacked modules:

1. Player
2. Equalizer
3. Playlist

The modules must be independent NSView-based components and must support later stacking, reordering, collapsing, detaching and resizing.

Visual style:

- dark navy/charcoal panels
- steel-blue borders
- subtle micro-bevels
- very small corner radii, approximately 2-4 px
- neon green #00ff32
- yellow #ffd21a
- orange #ff9d00
- black display areas
- Roboto Mono for ALL UI text
- no rounded modern SaaS cards
- no glassmorphism
- no SF Symbols
- no excessive shadows
- no large gradients
- no excessive animation

The UI should look like Winamp rendered at extremely high resolution rather than a completely new modern media player.

Use Core Graphics for the visual surfaces and controls.

All borders, sliders, buttons, EQ bands, EQ curve, spectrum, playlist rows, scrollbar and icons should be custom rendered.

Use point-based geometry and pixel snapping for Retina displays.

Preserve the compact proportions of the reference.

Player:

- compact spectrum display
- green digital timer
- track display
- bitrate
- sample rate
- mono/stereo
- seek and volume sliders
- EQ / PL controls
- previous/play/pause/stop/next/eject
- shuffle/repeat
- orange menu button

Equalizer:

- ON/AUTO
- EQ curve
- PRESETS
- preamp
- ten frequency bands
- +12/0/-12 dB scale
- custom vertical sliders

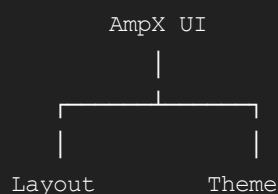
Playlist:

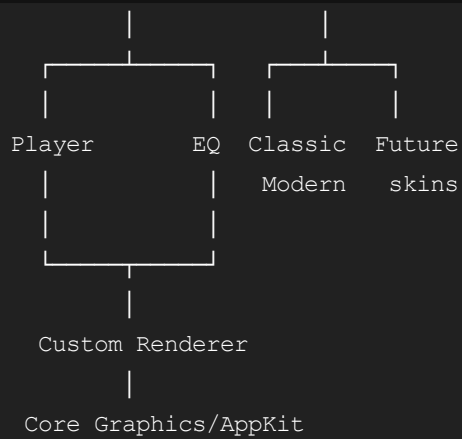
- black background
- green Roboto Mono text
- right-aligned durations
- rectangular blue-gray selection
- custom scrollbar
- ADD/REM/SEL/MISC
- mini transport
- time counter
- LIST OPTS

First implementation should prioritize visual fidelity over functionality. Build the complete static UI before connecting the audio engine.

One architectural choice I'd make very strongly

Since **AmpX** is ultimately a **Winamp restoration/fork** rather than a normal Mac media player, I'd make the rendering engine effectively a little **skin engine**:





Then the **Classic Modern** design we're working on becomes your first AmpX skin, rather than baking the colors and dimensions into every component.

That will pay off enormously when you eventually want to support actual Winamp `.wsz` skins or an AmpX-specific skin format.

And for this particular UI, **AppKit + Core Graphics is the route I'd choose over SwiftUI** if "looks exactly like the reference" is the priority.